

SubSub — Going to Production

A practical path from the current prototype to a live, multi-tenant product with iOS/Android apps.

Where you are today

`subsub.jsx` is ~4,000 lines of React in a single file, holding all state in memory via 63 `useState` hooks. Everything works, but nothing persists — reload and it resets to the seed data.

What's real: all the UI, the role system, the job/work-order model, matching and scoring logic, proximity math, coverage rules, the completion gate.

What's simulated: login, email/SMS, file uploads (filenames only), ZIP distances (built-in table of ~11 ZIPs), and persistence.

The good news is the data model is already well-shaped. Your `jobs`, `subs`, `users`, and `brand` objects map almost directly onto database tables.

Recommended stack

You said Cloudflare and GitHub, and for this app that's a genuinely good fit — it keeps everything on one platform with generous free tiers and no server management.

Layer	Choice	Why
Repo + CI	GitHub	Source of truth; Cloudflare deploys on push
Hosting	Cloudflare Pages	Static React build, global CDN, free SSL, preview deploys per branch
API	Cloudflare Workers	Same platform, no cold starts, runs at the edge
Database	Cloudflare D1 (SQLite)	Serverless SQL, zero config, plenty for this workload

File storage	Cloudflare R2	Insurance certs, bonds, signed WOs, measurement docs. No egress fees
Auth	Clerk or Supabase Auth	Don't build auth. Both handle magic links, sessions, password reset
Email	Resend or Postmark	The "request documents" emails
SMS	Twilio	The text-contractor feature
Geocoding	Google Geocoding API or US Census (free)	Replaces the ZIP table with real distances
Mobile	Capacitor	Wraps the existing React app as iOS/Android with minimal changes

Alternative worth considering: if you'd rather not hand-write SQL and auth glue, **Supabase** (Postgres + auth + storage + row-level security) gets you further faster, and you can still host the frontend on Cloudflare Pages. The tradeoff is two vendors instead of one. Given you're a small team shipping fast, I'd lean Supabase for the backend and Cloudflare for hosting — but D1 is the cleaner single-platform story if you prefer that.

Do you need a database?

Yes, unavoidably. Three reasons specific to this app:

1. **Two-sided data.** A contractor updates availability on their phone; the admin must see it. That requires shared server state.
2. **Compliance documents are legal artifacts.** Insurance certificates and signed agreements need durable storage with an audit trail, not browser memory.
3. **Work orders are contracts.** Once issued, a WO must be immutable and retrievable years later.

Schema

This maps directly from your current state objects. SQLite/D1 flavor; Postgres is nearly identical.

```
-- Multi-tenant root. One row per GC (outerhome.subsub.work)
```

```

CREATE TABLE tenants (
  id          TEXT PRIMARY KEY,
  name        TEXT NOT NULL,
  subdomain   TEXT UNIQUE NOT NULL,
  logo_key    TEXT,                -- R2 object key for the mark
  use_default_mark INTEGER DEFAULT 1,
  created_at  TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE users (
  id          TEXT PRIMARY KEY,
  tenant_id   TEXT NOT NULL REFERENCES tenants(id),
  auth_id     TEXT UNIQUE,         -- from Clerk/Supabase
  name        TEXT NOT NULL,
  email       TEXT NOT NULL,
  phone       TEXT,
  role        TEXT NOT NULL CHECK (role IN ('admin','pm','contractor')),
  contractor_id TEXT REFERENCES contractors(id), -- only for role='contractor'
  created_at  TEXT DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (tenant_id, email)
);

CREATE TABLE contractors (
  id          TEXT PRIMARY KEY,
  tenant_id   TEXT NOT NULL REFERENCES tenants(id),
  company     TEXT NOT NULL,
  contact     TEXT,
  phone       TEXT,
  email       TEXT,
  city        TEXT,
  state       TEXT,
  zip         TEXT,
  available   INTEGER DEFAULT 1,
  auto_schedule INTEGER DEFAULT 0,
  rating      REAL DEFAULT 0,      -- rolled up from crew ratings
  rated_jobs  INTEGER DEFAULT 0,
  accepted    INTEGER DEFAULT 0,
  declined    INTEGER DEFAULT 0,
  notes       TEXT,
  coverage_mode TEXT DEFAULT 'cities' CHECK (coverage_mode IN ('cities','radius'))
  created_at  TEXT DEFAULT CURRENT_TIMESTAMP
);

-- Trades + capabilities (many-to-many)
CREATE TABLE contractor_trades (
  contractor_id TEXT NOT NULL REFERENCES contractors(id) ON DELETE CASCADE,
  trade         TEXT NOT NULL,      -- roofing, siding, soffit_fascia, coping, ..

```

```

    PRIMARY KEY (contractor_id, trade)
);

CREATE TABLE contractor_capabilities (
    contractor_id TEXT NOT NULL REFERENCES contractors(id) ON DELETE CASCADE,
    capability     TEXT NOT NULL,
    PRIMARY KEY (contractor_id, capability)
);

-- Coverage: cities OR radii, per coverage_mode
CREATE TABLE contractor_cities (
    contractor_id TEXT NOT NULL REFERENCES contractors(id) ON DELETE CASCADE,
    city          TEXT NOT NULL,
    PRIMARY KEY (contractor_id, city)
);

CREATE TABLE contractor_radii (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    contractor_id TEXT NOT NULL REFERENCES contractors(id) ON DELETE CASCADE,
    zip             TEXT NOT NULL,
    miles           INTEGER NOT NULL,
    lat             REAL,                -- cached from geocoder
    lng             REAL
);

CREATE TABLE crews (
    id              TEXT PRIMARY KEY,
    contractor_id TEXT NOT NULL REFERENCES contractors(id) ON DELETE CASCADE,
    name            TEXT NOT NULL
);

CREATE TABLE crew_members (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    crew_id         TEXT NOT NULL REFERENCES crews(id) ON DELETE CASCADE,
    name            TEXT NOT NULL,
    role            TEXT
);

-- Days a contractor marked off
CREATE TABLE contractor_unavailable_days (
    contractor_id TEXT NOT NULL REFERENCES contractors(id) ON DELETE CASCADE,
    day           TEXT NOT NULL,        -- YYYY-MM-DD
    PRIMARY KEY (contractor_id, day)
);

-- Compliance documents (bond / insurance / agreement)
CREATE TABLE contractor_documents (
    id              TEXT PRIMARY KEY,
    contractor_id TEXT NOT NULL REFERENCES contractors(id) ON DELETE CASCADE,
    kind            TEXT NOT NULL CHECK (kind IN ('insurance','bond','contract')),

```

```

    file_key      TEXT NOT NULL,          -- R2 object key
    file_name     TEXT NOT NULL,
    expires_on    TEXT,                  -- worth adding: auto-flag expiring certs
    uploaded_by   TEXT REFERENCES users(id),
    uploaded_at   TEXT DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (contractor_id, kind)        -- one current doc per kind
);

-- THE JOB: every project fact lives here
CREATE TABLE jobs (
    id            TEXT PRIMARY KEY,
    tenant_id     TEXT NOT NULL REFERENCES tenants(id),
    title         TEXT NOT NULL,
    client        TEXT,
    address       TEXT,
    city          TEXT,
    zip           TEXT,
    lat           REAL,
    lng           REAL,
    sqft          INTEGER,
    stories       INTEGER,
    start_date    TEXT,
    start_time    TEXT DEFAULT '07:00',
    scope         TEXT,
    material_source TEXT,
    materials_paid_by TEXT,
    status        TEXT DEFAULT 'active' CHECK (status IN ('active','completed'))
    completed_at  TEXT,
    notes         TEXT,
    created_by    TEXT REFERENCES users(id),
    created_at    TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE job_trades (
    job_id        TEXT NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,
    trade         TEXT NOT NULL,
    PRIMARY KEY (job_id, trade)
);

CREATE TABLE job_measurement_docs (
    id            TEXT PRIMARY KEY,
    job_id        TEXT NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,
    file_key      TEXT NOT NULL,
    file_name     TEXT NOT NULL,
    uploaded_by   TEXT REFERENCES users(id),
    uploaded_at   TEXT DEFAULT CURRENT_TIMESTAMP
);

```

```

-- THE WORK ORDER: one per trade per job, generated on assignment.
-- Immutable once issued; changes create a new WO.
CREATE TABLE work_orders (
  id          TEXT PRIMARY KEY,
  wo_number   TEXT UNIQUE NOT NULL,    -- WO-1234
  job_id      TEXT NOT NULL REFERENCES jobs(id),
  trade       TEXT NOT NULL,
  contractor_id TEXT NOT NULL REFERENCES contractors(id),
  crew_id     TEXT REFERENCES crews(id),
  trade_scope TEXT,
  value_cents INTEGER,                -- store money as integer cents
  status      TEXT DEFAULT 'pending'
              CHECK (status IN ('pending', 'accepted', 'declined')),
  auto_scheduled INTEGER DEFAULT 0,
  rating      INTEGER,                -- 1-5, only after job completed
  signed_file_key TEXT,
  issued_at   TEXT DEFAULT CURRENT_TIMESTAMP,
  responded_at TEXT,
  UNIQUE (job_id, trade)              -- one contractor per trade per job
);

-- Append-only audit log. You will want this.
CREATE TABLE events (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  tenant_id   TEXT NOT NULL,
  actor_id    TEXT,
  kind        TEXT NOT NULL,          -- wo.issued, wo.accepted, job.completed, doc.uploa
  subject_id  TEXT,
  payload     TEXT,                  -- JSON
  created_at  TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_jobs_tenant_status ON jobs(tenant_id, status);
CREATE INDEX idx_wo_contractor ON work_orders(contractor_id, status);
CREATE INDEX idx_wo_job ON work_orders(job_id);
CREATE INDEX idx_contractors_tenant ON contractors(tenant_id);

```

Three schema decisions worth calling out

- **Money as integer cents.** `value_cents INTEGER`, never a float. Your current code stores raw digit strings, which is fine, but the DB should be cents.
- **Work orders are immutable.** Don't `UPDATE` scope or value after issuing. To change terms, void and reissue. This is what makes them contracts.

- **Cache lat/lng.** Geocode the job address and each radius ZIP once, store the coordinates, then distance math is pure SQL — no API call per search.

Verifying WA contractor licenses

Yes — and it's free. Washington publishes the entire L&I contractor registry as open data on data.wa.gov via Socrata's SODA API. No key, no scraping, no vendor. Four datasets join on the license number:

Dataset	ID	What it gives you
General	m8qx-ubtq	Status, license type, effective/expiration dates, UBI, address, primary principal
Insurance	ciwg-agsx	Carrier, policy number, coverage amount , effective/expiration/cancel dates
Surety bond	bzff-4fmt	Surety firm, bond amount , impairment status, dates
Principals	4xk5-x9j6	Owners and officers of record

The registry holds ~160,000 licenses (~76,000 active) and L&I refreshes it on business days.

The call

```
const LNI = "https://data.wa.gov/resource";

async function verifyLicense(licenseNumber) {
  const q = `?contractorlicensenum=${encodeURIComponent(licenseNumber)}`;
  const [general, insurance, bond] = await Promise.all([
    fetch(`${LNI}/m8qx-ubtq.json${q}`).then((r) => r.json()),
    fetch(`${LNI}/ciwg-agsx.json${q}`).then((r) => r.json()),
    fetch(`${LNI}/bzff-4fmt.json${q}`).then((r) => r.json()),
  ]);

  const lic = general[0];
  if (!lic) return { found: false };

  const today = new Date().toISOString().slice(0, 10);
```

```

const current = (rows, endField) =>
  rows.filter((r) => !r.cancel_date && (!r[endField] || r[endField].slice(0, 10)
    .sort((a, b) => (b.effective_date || "").localeCompare(a.effective_date |

return {
  found: true,
  businessName: lic.businessname,
  status: lic.contractorlicensestatus,          // ACTIVE / EXPIRED / SUSPENDED
  licenseType: lic.contractorlicensetypecodedesc,
  ubi: lic.ubi,
  effectiveDate: lic.licenseeffectivedate?.slice(0, 10),
  expirationDate: lic.licenseexpirationdate?.slice(0, 10),
  suspendDate: lic.contractorlicensesuspenddate?.slice(0, 10) || null,
  principal: lic.primaryprincipalname,
  insurance: current(insurance, "insurance_expiration_date"),
  bond: current(bond, "bond_expiration_date"),
};
}

```

Field names in the addenda datasets vary slightly — hit each endpoint once and log a row before you map them.

Three things worth doing with it

1. Cross-check the certificate against the state. This is the real value. A contractor uploads a COI claiming \$1M coverage; L&I independently reports the carrier, policy number, and coverage amount on file. If they disagree, you're looking at a stale or altered certificate. Same for the bond amount. Flag the mismatch for the reviewer rather than auto-failing — L&I lags a few days behind a renewal.

2. Nightly sweep. Registrations expire and get suspended without anyone telling you. A cron over your active contractors, re-running the lookup and flagging any status change, catches a lapsed registration before you dispatch someone on it. Pair it with the `expires_on` sweep on stored documents.

3. Autofill onboarding. Ask for the license number first, then pull business name, address, UBI, and principal from the state rather than making the contractor type it. Fewer fields for someone filling this in on a phone.

Caveats

- L&I's own disclaimer: the database updates weekday mornings, so a same-day renewal may not appear yet. Don't hard-fail a contractor on a single miss — surface it for review.
- Workers' comp is a **separate** L&I system. The contractor registry tells you a registration

is active; it doesn't confirm a paid-to-date workers' comp account. L&I's public Verify tool shows both, and there's a separate industrial-insurance lookup if you want to automate it.

- Registration $\neq$ qualification. Active status means bonded, insured, and registered — not that they do good work. Your rating system covers that.
- Commercial wrappers exist (Apify and similar) that pre-join these datasets for a few cents per record. Worth it only if you don't want to maintain the join yourself; the underlying data is the same and free.

Schema addition

```
ALTER TABLE contractors ADD COLUMN license_number TEXT;
ALTER TABLE contractors ADD COLUMN ubi TEXT;

CREATE TABLE license_checks (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  contractor_id   TEXT NOT NULL REFERENCES contractors(id) ON DELETE CASCADE,
  status          TEXT,                -- ACTIVE / EXPIRED / SUSPENDED / NOT_FOUND
  license_type    TEXT,
  effective_date  TEXT,
  expiration_date TEXT,
  suspend_date    TEXT,
  bond_amount_cents  INTEGER,
  bond_surety      TEXT,
  insurance_coverage_cents INTEGER,
  insurance_carrier TEXT,
  raw             TEXT,                -- full JSON response
  checked_at      TEXT DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_license_checks_contractor ON license_checks(contractor_id, checked_at);
```

Keep every check rather than overwriting one row — when a dispute turns on whether someone was registered on the day they worked, the history is the answer.

The identity model: one contractor, many hiring companies

This is the most consequential architectural decision in the product. **The prototype now implements it**, so the server work is a direct translation rather than a redesign.

In `subsub.jsx`, look for the `DATA MODEL` block: `COMPANY_FIELDS / ENGAGEMENT_FIELDS` is the field-ownership map, `splitPatch()` routes writes to the right table, and `composeSub()`

flattens a company + engagement back into the object the UI reads. Those three functions are the whole design, and they port to the API layer almost unchanged.

The demo data exercises it: **Outerhome** (Basic) engages seven companies, **Harbor Point Builders** (Scale) engages three of the same ones. Cascade Roofworks is one company record with two engagements — Outerhome rates them 4.8 and has verified their insurance; Harbor Point rates them 4.4 and still has it awaiting review. Same licence, same crews, same uploaded file, two verdicts. Sign in as Miguel (miguel@cascaderooft.com) and the account switcher shows both.

The problem

The prototype treats a contractor as a row belonging to one tenant — `contractors` has a `tenant_id`. That breaks on two facts about the real world:

1. **A subcontractor works for several general contractors at once.** Cascade Roofworks might sub for five GCs. Under the current model they get onboarded five times, upload the same certificate of insurance five times, and maintain five separate profiles. Each GC reviews the same document independently. That's the exact busywork the product exists to remove, and it's the thing that would make subs resent using it.
2. **A subcontractor is often also a hiring company.** A siding sub with four crews subs out their own gutter work. They need to be a contractor *in someone else's account* and an admin *in their own* — same person, same login, two roles.

The fix: separate the company from the relationship

Three concepts instead of one:

- **companies** — a business that exists in the world. Global, not owned by any tenant. Identified by its WA L&I licence number and UBI, which is what makes dedup possible.
- **accounts** — a hiring company's workspace (what the prototype calls a tenant). Has a plan, users, jobs, branding.
- **engagements** — the link between an account and a company, carrying everything relationship-specific: this GC's rating of them, their pay history, their auto-schedule setting, the trades they cover *for this GC*.

```
-- A business in the world. One row no matter how many GCs use them.
```

```
CREATE TABLE companies (  
  id                TEXT PRIMARY KEY,  
  legal_name        TEXT NOT NULL,  
  license_number    TEXT,                -- WA L&I; the natural dedup key  
  ubi               TEXT,
```

```

    phone            TEXT,
    email            TEXT,
    mail_street      TEXT,
    mail_city        TEXT,
    mail_state       TEXT,
    mail_zip         TEXT,
    created_at       TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE UNIQUE INDEX idx_companies_license ON companies(license_number)
    WHERE license_number IS NOT NULL;

-- A hiring company's workspace.
CREATE TABLE accounts (
    id                TEXT PRIMARY KEY,
    company_id       TEXT REFERENCES companies(id), -- the hiring company itself
    name             TEXT NOT NULL,
    subdomain        TEXT UNIQUE,
    plan             TEXT NOT NULL DEFAULT 'basic',
    logo_key         TEXT,
    created_at       TEXT DEFAULT CURRENT_TIMESTAMP
);

-- The relationship. This is where per-GC data lives.
CREATE TABLE engagements (
    id                TEXT PRIMARY KEY,
    account_id       TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
    company_id       TEXT NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
    status           TEXT NOT NULL DEFAULT 'invited', -- invited/active/paused/ended
    auto_schedule    INTEGER NOT NULL DEFAULT 0,
    rating           REAL,
    rated_jobs       INTEGER NOT NULL DEFAULT 0,
    accepted_count   INTEGER NOT NULL DEFAULT 0,
    declined_count   INTEGER NOT NULL DEFAULT 0,
    notes            TEXT,
    invited_at       TEXT DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (account_id, company_id)
);

```

What sits where

The rule: if it's a fact about the business, it hangs off `companies` . If it's a fact about the relationship, it hangs off `engagements` .

Data	Belongs to	Why
Licence number,		

UBI, legal name	companies	One business, one licence
Licence check results	companies	The state's answer is the same for everyone
Crews and crew members	companies	Their crews are their crews
Crew availability / off days	companies	A crew booked by GC A is not available to GC B
Insurance, bond, agreement files	companies	One certificate, uploaded once
Document approvals	engagements	Each GC approves against <i>their own</i> requirements
Trades and capabilities offered	engagements	May do roofing for one GC, gutters for another
Coverage area	companies (default), overridable per engagement	Usually the same everywhere
Rating, accept rate, earnings	engagements	Your experience of them is yours
Auto-schedule setting	engagements	A trust setting, per relationship
Uniform orders	engagements	Branded to one specific GC

The crew-availability line is the subtle one. A crew physically cannot be on two GCs' jobs the same day, so availability has to be global to the company — otherwise you'll double-book people. This means `dayStatus()` must check work orders across *all* accounts that engage the company, while only showing the requesting account the details of its own jobs. Everything else stays scoped to the account.

Document approvals per engagement is the other important one. GC A requires \$2M aggregate; GC B accepts \$1M. The same uploaded certificate gets a different verdict from each. So the file lives once on `companies`, and the approval — status, the ticked checks, the recorded limits, any accepted shortfall and its reason — lives per engagement:

```
CREATE TABLE company_documents (      -- the file, uploaded once
  id          TEXT PRIMARY KEY,
  company_id  TEXT NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
```

```

kind            TEXT NOT NULL,          -- insurance / bond / agreement
file_key        TEXT NOT NULL,          -- R2 object key
filename        TEXT,
uploaded_at     TEXT DEFAULT CURRENT_TIMESTAMP,
expires_on      TEXT
);

CREATE TABLE document_reviews (        -- one GC's verdict on that file
  id            TEXT PRIMARY KEY,
  engagement_id TEXT NOT NULL REFERENCES engagements(id) ON DELETE CASCADE,
  document_id   TEXT NOT NULL REFERENCES company_documents(id) ON DELETE CASCADE,
  status        TEXT NOT NULL,          -- pending / verified / rejected
  limits        TEXT,                  -- JSON: per-line coverage amounts
  checks        TEXT,                  -- JSON: which confirmations were ticked
  overrides     TEXT,                  -- JSON: {line: reason} accepted below req
  note          TEXT,
  reviewed_by   TEXT REFERENCES users(id),
  reviewed_at   TEXT,
  UNIQUE (engagement_id, document_id)
);

```

Users and roles

A user belongs to a person, not a tenant. Membership is the join, and one user can hold different roles in different accounts:

```

CREATE TABLE users (
  id            TEXT PRIMARY KEY,
  email         TEXT UNIQUE NOT NULL,   -- their identity; also their username
  name          TEXT NOT NULL,
  phone         TEXT,
  notify_email  INTEGER NOT NULL DEFAULT 1,
  notify_sms    INTEGER NOT NULL DEFAULT 0,
  created_at    TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE memberships (
  id            TEXT PRIMARY KEY,
  user_id       TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  account_id    TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  role          TEXT NOT NULL,          -- admin / pm / contractor
  company_id    TEXT REFERENCES companies(id), -- set when role = contractor
  UNIQUE (user_id, account_id)
);

```

So Miguel at Cascade Roofworks signs in once and sees an account switcher: *contractor* in five GCs' accounts, *admin* in his own. Only `admin` and `pm` memberships consume a seat — contractor memberships are free and unlimited, which is what makes the free-contractor-login promise work.

Invite flow with dedup

When a GC invites a subcontractor, match before creating:

1. Licence number given → look up `companies.license_number`. Hit means the business already exists.
2. No licence → match on email or normalised legal name, and confirm with the user rather than silently merging.
3. Existing company → create an `engagement` only. Their profile, crews and documents are already there, so the GC sees them immediately and only needs to run their own approval.
4. New company → create `companies` + `engagement` + an invite.

This is a feature, not just hygiene. A sub already on SubSub for another GC gets added in one step with their paperwork already uploaded — that's the strongest possible onboarding experience, and it compounds as the network grows. Worth surfacing in the invite UI:
"Cascade Roofworks is already on SubSub. They'll appear in your account once they accept — you'll just need to approve their documents against your requirements."

Plan limits under this model

Limits are counted per account, on engagements rather than companies:

- **Contractors** = `SELECT count(*) FROM engagements WHERE account_id = ? AND status = 'active'`
- **Users** = memberships where role in ('admin','pm')
- **Jobs** = `SELECT count(*) FROM jobs WHERE account_id = ? AND created_at >= date('now','start of month')`

The monthly job count must come from the server, not a client-side filter on loaded rows — otherwise a stale client under-counts and the cap leaks.

Porting the prototype to SQL

The prototype's in-memory tables map 1:1 onto the schema above — `companies`, `engagements`, `accounts`, `memberships`, `users`, `jobs` (with `account_id`). Three things to

carry across that the prototype already gets right:

1. **`splitPatch()` becomes your write path.** Any endpoint that updates a contractor should route fields by the same ownership map, rather than letting callers write wherever.
2. **Crew availability queries must not be account-scoped.** The prototype passes `allJobs` into `dayStatus()` for exactly this reason, and reports “booked on another company’s job” without leaking whose. In SQL that’s a work-order query across all accounts, joined on company, with only the *fact* of the booking returned.
3. **Dedupe at invite time, on `UPPER(TRIM(license_number))`.** The prototype’s `addSub()` does this and reuses the existing company row, creating only the engagement.

If you’d rather start from the older single-table shape, the mechanical migration is:

```
INSERT INTO companies (id, legal_name, license_number, ubi, phone, email, ...)
  SELECT id, company, license_number, ubi, phone, email, ... FROM contractors;
```

```
INSERT INTO engagements (id, account_id, company_id, auto_schedule, rating, rated
  SELECT lower(hex(randomblob(16))), tenant_id, id, auto_schedule, rating, rated_
  FROM contractors;
```

Then dedup on licence number, since the same business may exist as several rows across tenants. Do this before launch: merging company records after documents, approvals and ratings have accumulated against duplicates is considerably harder.

Getting it live

1. Repo and project structure

```
mkdir subsub && cd subsub
npm create vite@latest . -- --template react
npm install lucide-react
git init && git remote add origin git@github.com:YOURORG/subsub.git
```

Split the monolith as you go — don’t do it all at once:

```
src/
  App.jsx
  lib/          api.js, money.js, coverage.js, matching.js
  components/   JobForm, WorkOrderDoc, PickContractor, ...
```

```
views/           Dashboard, Contractors, Jobs, Availability, Portal, Settings
worker/
  index.js       Hono routes
  schema.sql
```

Pull the pure functions out first — `coversZip`, `zipDistance`, `formatMoney`, `issueWO`, the scoring math. They have no React dependency and are the easiest things to unit-test.

2. Database

```
npm i -g wrangler && wrangler login
wrangler dl create subsub-db
wrangler dl execute subsub-db --file=./worker/schema.sql
wrangler r2 bucket create subsub-files
```

Add the bindings to `wrangler.toml`:

```
name = "subsub-api"
main = "worker/index.js"
compatibility_date = "2026-01-01"

[[d1_databases]]
binding = "DB"
database_name = "subsub-db"
database_id = "..."

[[r2_buckets]]
binding = "FILES"
bucket_name = "subsub-files"
```

3. API

Use Hono — it's tiny and built for Workers:

```
import { Hono } from 'hono';
const app = new Hono();

app.get('/api/jobs', async (c) => {
  const { tenantId } = c.get('auth');
  const { results } = await c.env.DB
    .prepare('SELECT * FROM jobs WHERE tenant_id = ? ORDER BY created_at DESC')
    .bind(tenantId).all();
  return c.json(results);
});
```

```
// Assigning a contractor issues the work order — server-side, atomic
app.post('/api/jobs/:id/assign', async (c) => {
  const { role, tenantId, userId } = c.get('auth');
  if (role === 'contractor') return c.json({ error: 'forbidden' }, 403);
  const { trade, contractorId, tradeScope, valueCents } = await c.req.json();

  const docs = await c.env.DB.prepare(
    `SELECT COUNT(*) n FROM contractor_documents WHERE contractor_id = ?`
  ).bind(contractorId).first();
  if (docs.n < 3) return c.json({ error: 'documents_incomplete' }, 409);

  const wo = `WO-${Math.floor(1000 + Math.random() * 9000)}`;
  await c.env.DB.prepare(
    `INSERT INTO work_orders
      (id, wo_number, job_id, trade, contractor_id, trade_scope, value_cents, stat
    VALUES (?, ?, ?, ?, ?, ?, ?,
      (SELECT CASE WHEN auto_schedule = 1 THEN 'accepted' ELSE 'pending' END
      FROM contractors WHERE id = ?),
      (SELECT auto_schedule FROM contractors WHERE id = ?))`
  ).bind(crypto.randomUUID(), wo, c.req.param('id'), trade, contractorId,
    tradeScope, valueCents, contractorId, contractorId).run();

  return c.json({ wo });
});

export default app;
```

Enforce roles on the server. Right now `ROLES[role].can` gates the UI, which is the correct UX but no security at all — a contractor could call any endpoint directly. Every handler needs the check.

4. File uploads

Never route files through the Worker. Issue a presigned R2 URL, let the browser upload directly, then record the key:

```
app.post('/api/uploads/sign', async (c) => {
  const { kind, fileName } = await c.req.json();
  const key = `${c.get('auth').tenantId}/${kind}/${crypto.randomUUID()}-${fileName}`;
  const url = await c.env.FILES.createPresignedUrl(key, { method: 'PUT', expiresI
  return c.json({ url, key });
});
```

5. Deploy

```
wrangler deploy                                # the API
# Pages: connect the GitHub repo in the dashboard
#   build command: npm run build
#   output dir:    dist
```

Then in Cloudflare DNS add a wildcard `*.subsub.work` CNAME to your Pages project. Read the subdomain from `location.hostname` on boot, look up the tenant, and load that brand — which is exactly the shape your `brand` state already has.

Every push to `main` deploys; every PR gets a preview URL.

Mobile apps

Capacitor wraps the same React build, so you're not rewriting anything:

```
npm i @capacitor/core @capacitor/cli && npx cap init
npm i @capacitor/ios @capacitor/android
npx cap add ios && npx cap add android
npm run build && npx cap sync
npx cap open ios          # needs Xcode + a Mac
```

The responsive work from earlier rounds is what makes this viable. Two additions worth making native:

- **Push notifications** (`@capacitor/push-notifications`) — a contractor should get a push when a work order lands, not an email they miss.
- **Camera** (`@capacitor/camera`) — let contractors photograph an insurance certificate rather than finding a scanner. This will meaningfully improve your compliance rate.

Both stores will want a privacy policy and account deletion. Apple review typically takes a few days for a first submission.

Suggested order of work

Phase 1 — persistence (1–2 weeks). Schema, auth, CRUD for contractors and jobs. Replace `useState` with fetches. Nothing new for users, but everything stops disappearing.

Phase 2 — files and email (1 week). R2 uploads for documents, Resend for the request-documents flow. This is where the compliance loop starts actually working.

Phase 3 — real geography (a few days). Geocode on save, store lat/lng, replace the ZIP table. Your matching gets accurate for any address instead of the eleven ZIPs currently in the table.

Phase 4 — multi-tenant (1 week). Subdomain routing, tenant onboarding, per-tenant branding from the DB.

Phase 5 — mobile (1–2 weeks). Capacitor wrap, push, camera, store submissions.

Rough monthly cost

	Starting out	~50 GCs
Cloudflare Pages	\$0	\$0
Workers	\$0	\$5
D1	\$0	~\$5
R2	\$0	~\$5
Auth (Clerk)	\$0	~\$25
Email (Resend)	\$0	~\$20
Twilio SMS	pay per message	~\$20
Apple + Google developer	\$99/yr + \$25 once	same

Call it **\$0–10/month** to start and **under \$100/month** at fifty contractors. The infrastructure is not going to be your constraint.

Two things to get right early

Document expiration. I added `expires_on` to `contractor_documents` because insurance certificates lapse, and a contractor who was compliant in March may not be in September. A nightly job that flags certificates expiring within 30 days — and fires your existing request-documents email automatically — turns a recurring chore into something the system handles.

Work order immutability. Resist the urge to let admins edit an issued work order. Void and

reissue instead. The day there's a dispute about what a contractor agreed to, an append-only trail is worth more than the convenience.